

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

Departamento de Computación, Electrónica y Mecatrónica

Reporte de práctica

Práctica 0: Descripción de Circuitos Combinacionales y Secuenciales en VHDL

Materia	P26-LIS2022-1 Arquitecturas Computacionales
Profesor	Dr. Omar Guillén Fernández
Sección	2
Periodo	Primavera 2026
Fecha de práctica	19 de enero de 2026
Fecha de entrega	9 de febrero de 2026

Integrantes

Robbie Nicolás Curioso de Salazar	186028	robbie.curiosodr@udlap.mx
Hector López Sanchez	185979	hector.lopezsz@udlap.mx

1. Introducción

En esta práctica se describieron circuitos combinacionales y secuenciales usando VHDL. Se hicieron módulos básicos: medio sumador, sumador completo, sumador de 4 bits, sumador de n bits con signo, flip-flop tipo D y un registro PIPO. Se verificó el funcionamiento con simulación en Active-HDL.

Por falta de tiempo y por indicaciones del tutor, sólo se alcanzó a llegar hasta la implementación en hardware del medio sumador usando Vivado y la tarjeta Pynq Z1. Los demás módulos quedaron verificados en simulación.

2. Objetivos

- Describir circuitos digitales básicos en VHDL.
- Simular en Active-HDL y validar resultados con tablas y casos de prueba.
- Leer diagramas de tiempo para confirmar el comportamiento.
- Implementar el medio sumador en Vivado y comprobar entradas y salidas en la Pynq Z1.

3. Materiales y herramientas

- Software: Active-HDL Student Edition y Vivado 2020.1 o superior.
- Hardware: Tarjeta Pynq Z1 con switches y LEDs.

- Computadora y cable USB para programar la tarjeta.

4. Marco teórico

Un circuito combinacional depende sólo de las entradas actuales. Un circuito secuencial guarda estado y normalmente se controla con reloj.

En el medio sumador se cumple:

$$Sum = A \oplus B, \quad C = A \wedge B$$

En el sumador completo se cumple:

$$Sum = A \oplus B \oplus Cin$$

$$C_s = (A \wedge B) \vee (Cin \wedge (A \oplus B))$$

Para trabajar con números con signo se usa complemento a dos. En VHDL es mejor usar `numeric_std` y el tipo `signed` para evitar errores al sumar vectores.

5. Diseño y procedimiento

5.1. Simulación en Active-HDL

1. Se creó el proyecto y se agregaron los archivos .vhd.
2. Se compiló para verificar que no hubiera errores.
3. Se creó el archivo de forma de onda y se aplicaron estímulos a las entradas.
4. Se ejecutó la simulación y se comparó con lo esperado.

5. Se tomaron capturas como evidencia.

5.2. Implementación en hardware con Vivado y Pynq Z1

1. Se creó el proyecto en Vivado y se dejó `half_adder` como módulo principal.
2. Se asignaron pines de switches y LEDs en el archivo de restricciones.
3. Se corrió síntesis e implementación y se generó el bitstream.
4. Se programó la tarjeta y se probaron las cuatro combinaciones.

6. Resultados y discusión

Aquí se deja todo en orden: primero la función, luego la evidencia (simulación), y luego el código. En el caso del medio sumador, también se agrega la evidencia de la FPGA.

6.1. Módulo 1: Medio sumador half_adder

Función.

$$Sum = A \oplus B, \quad C = A \wedge B$$

Simulación.

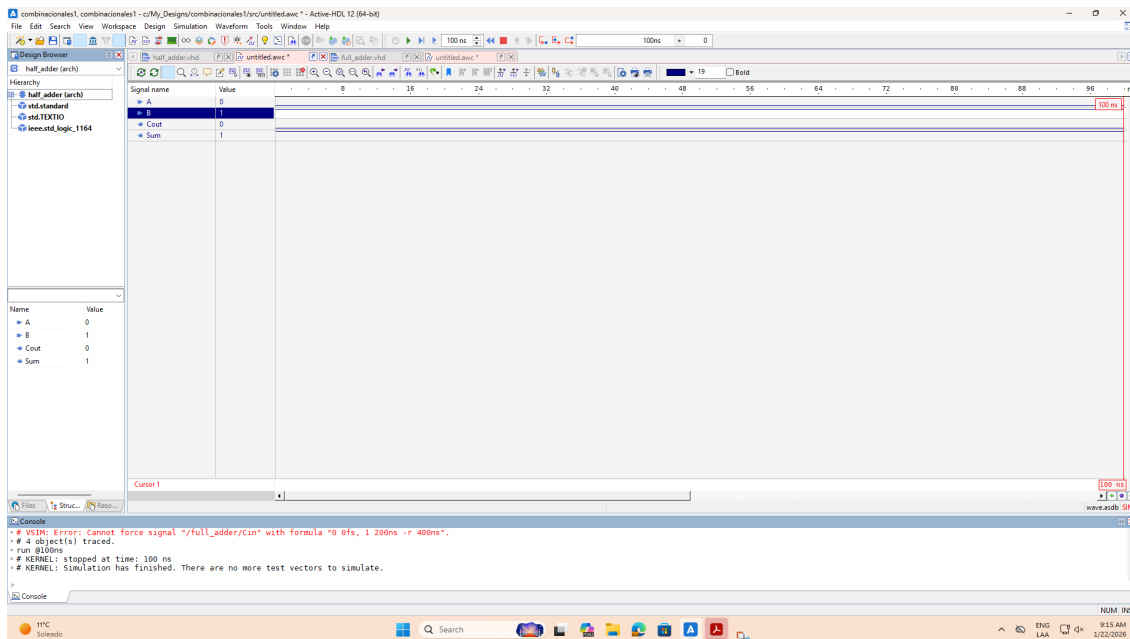


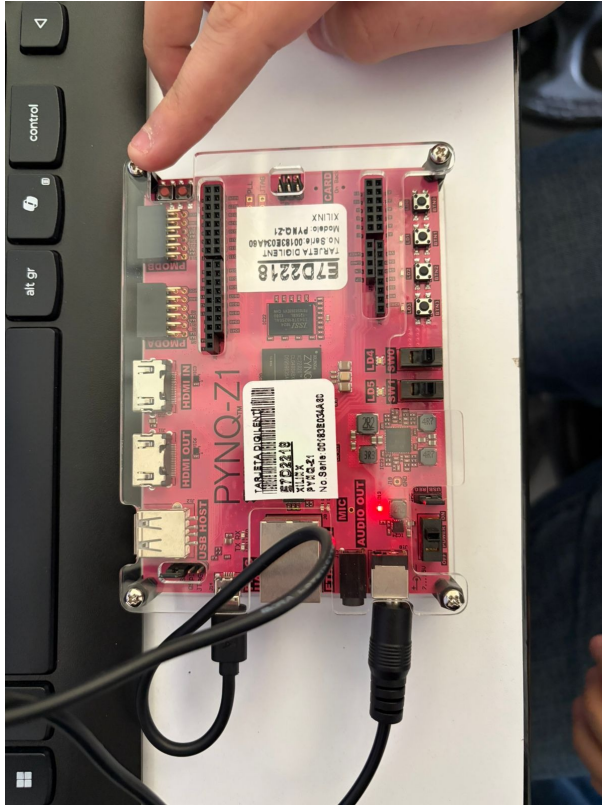
Figura 1: Simulación del medio sumador en Active-HDL.

Verificación.

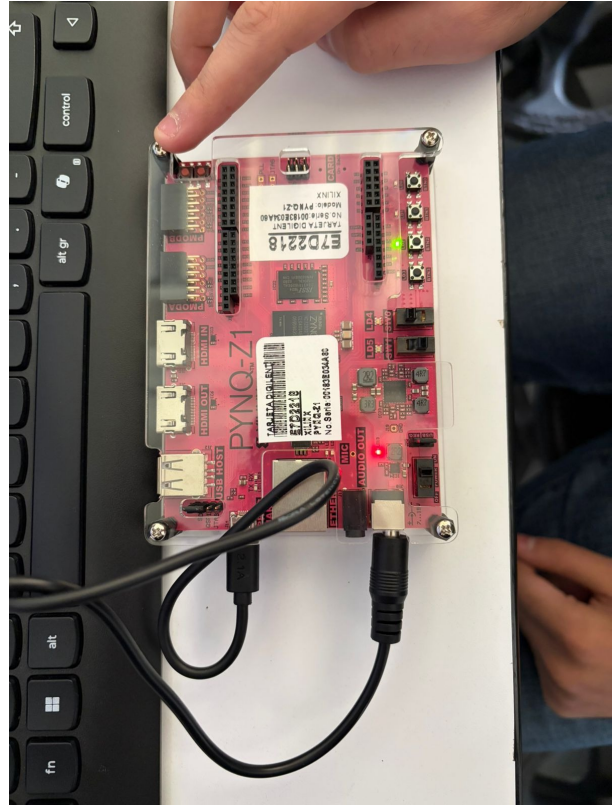
Cuadro 1: Tabla de verificación del medio sumador.

A	B	Sum	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementación en FPGA (Pynq Z1).



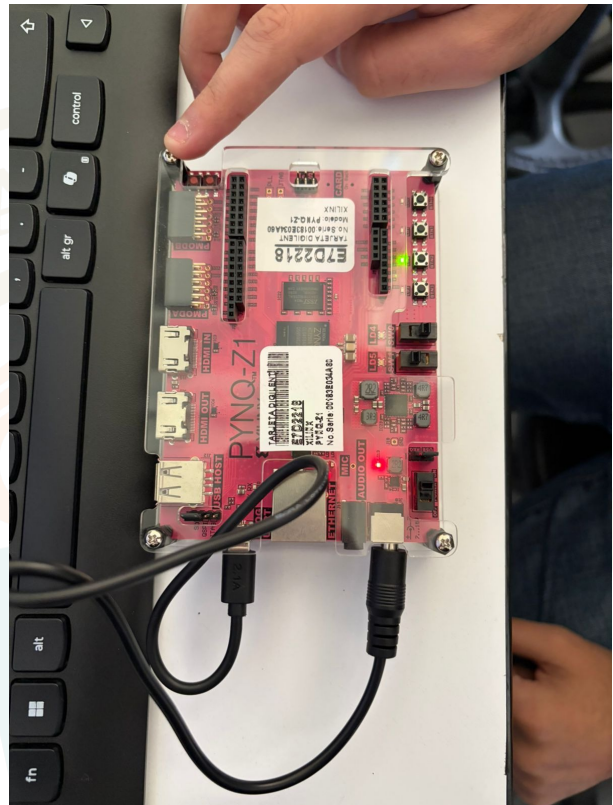
(a) Caso 1



(b) Caso 2



(c) Caso 3



(d) Caso 4

Figura 2: Evidencias del medio sumador en la Pynq Z1.

186028; robbie.curiosodr@udlap.mx

Código VHDL.

```

1  --Librería:
2  Library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4  --Entidad:
5  entity half_adder is
6      port (
7          A, B : in std_logic; --std_logic=1 bit
8          Cout, Sum : out std_logic
9      );
10 end half_adder;
11 --Arquitectura:
12 architecture arch of half_adder is
13 begin
14     Sum <= A xor B; --<=: Asignación
15     Cout <= A and B;
16 end arch;

```

Listing 1: Código VHDL del medio sumador.

6.2. Módulo 2: Sumador completo full_adder

Función.

$$Sum = A \oplus B \oplus Cin$$

$$C_s = (A \wedge B) \vee (Cin \wedge (A \oplus B))$$

Simulación.

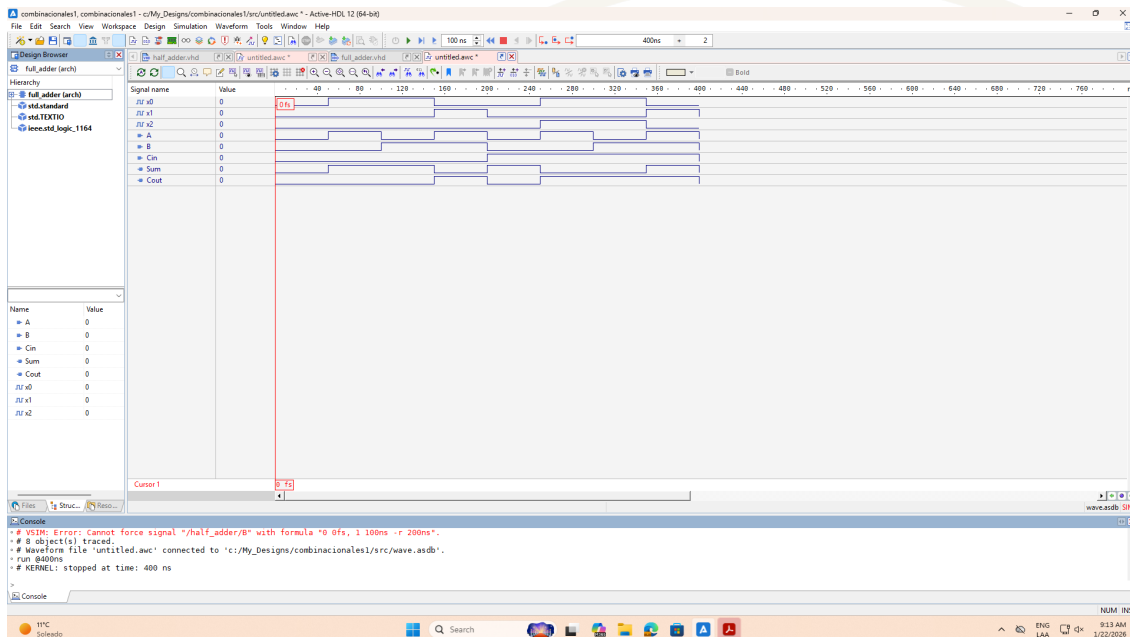


Figura 3: Simulación del sumador completo en Active-HDL.

Código VHDL.

```

1  Library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  Entity full_adder is
4      port(
5          A, B, Cin : in std_logic;
6          Sum, Cout : out std_logic
7      );
8  end full_adder;
9
10 architecture arch of full_adder is
11     signal x0, x1, x2: std_logic; --A adimos esto para mapear correctamente (x1->c1 x0->suma)
12 begin
13     HA1: entity work.half_adder(arch) port map (A, B, x1, x0);
14     HA2: entity work.half_adder(arch) port map (x0, Cin, x2, Sum);
15     Cout <= x1 or x2;
16 end arch;

```

Listing 2: Código VHDL del sumador completo.

6.3. Módulo 3: Sumador de 4 bits adder_4bits

Función. Suma dos números de 4 bits y entrega un acarreo final C_4 .

Simulación.

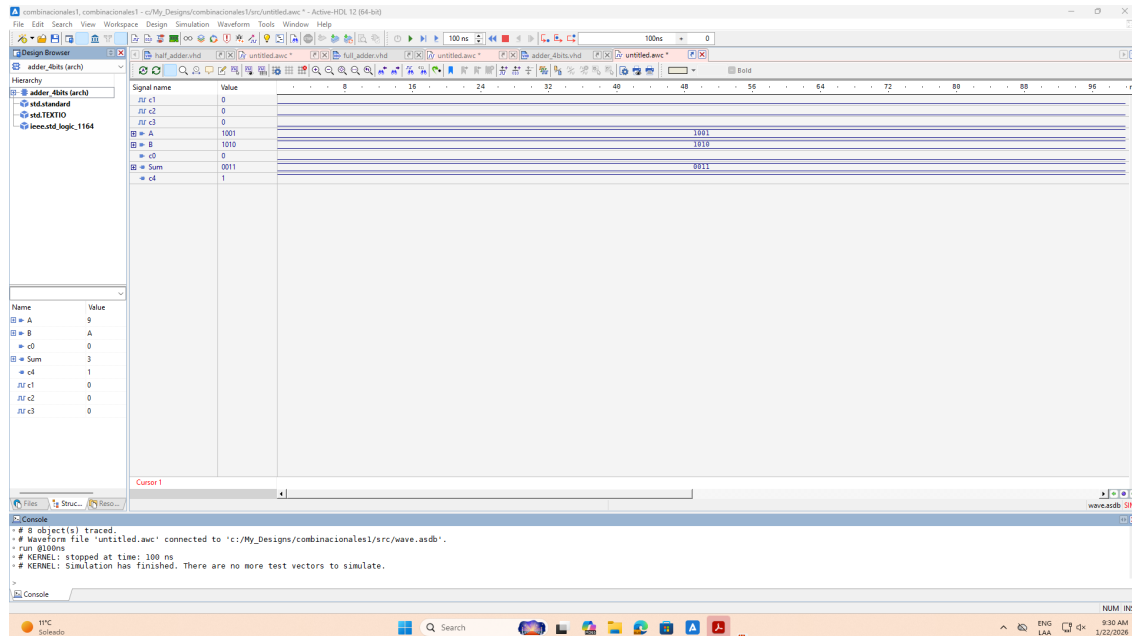


Figura 4: Simulación del sumador de 4 bits en Active-HDL.

Casos medidos.

Cuadro 2: Casos de prueba del sumador de 4 bits.

A	B	S	C_4
0000	0000	0000	0
0001	0001	0010	0
0011	0010	0101	0
0111	0001	1000	0
1111	0001	0000	1
1111	1111	1110	1

Código VHDL.

```

1  Library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity adder_4bits is
5      port(
6          A, B : in  std_logic_vector(3 downto 0);
7          c0   : in  std_logic;
8          Sum  : out std_logic_vector(3 downto 0);
9          c4   : out std_logic
10     );
11 end adder_4bits;
12
13 architecture arch of adder_4bits is
14     signal c1, c2, c3 : std_logic;
15 begin
16     FA0: entity work.full_adder(arch) port map (A(0), B(0), c0, Sum(0), c1);
17     FA1: entity work.full_adder(arch) port map (A(1), B(1), c1, Sum(1), c2);
18     FA2: entity work.full_adder(arch) port map (A(2), B(2), c2, Sum(2), c3);
19     FA3: entity work.full_adder(arch) port map (A(3), B(3), c3, Sum(3), c4);
20 end arch;

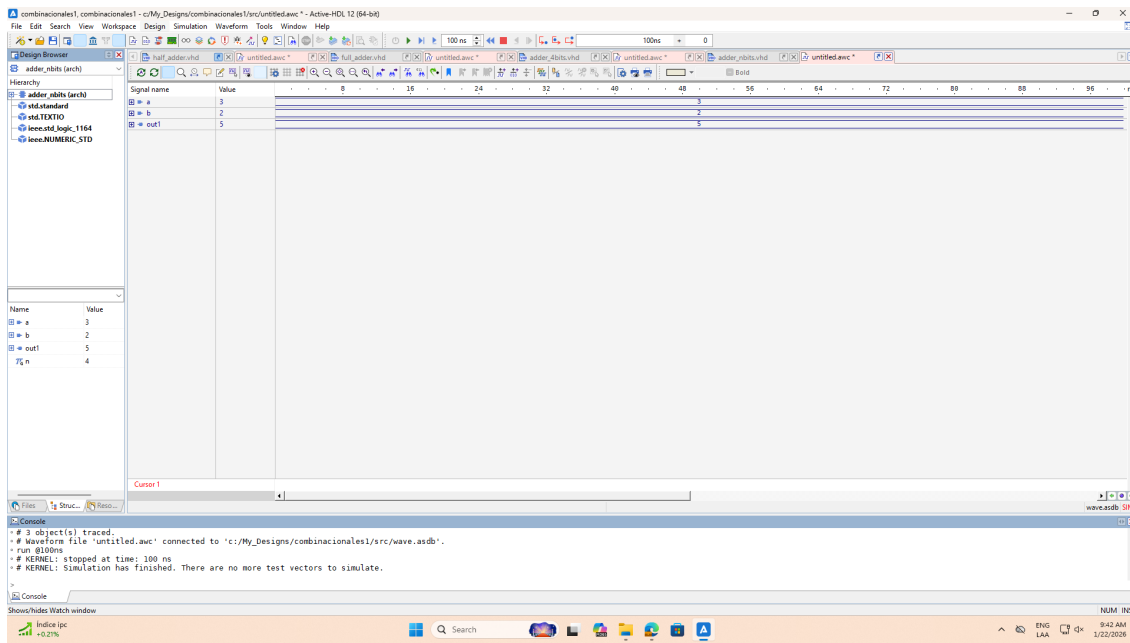
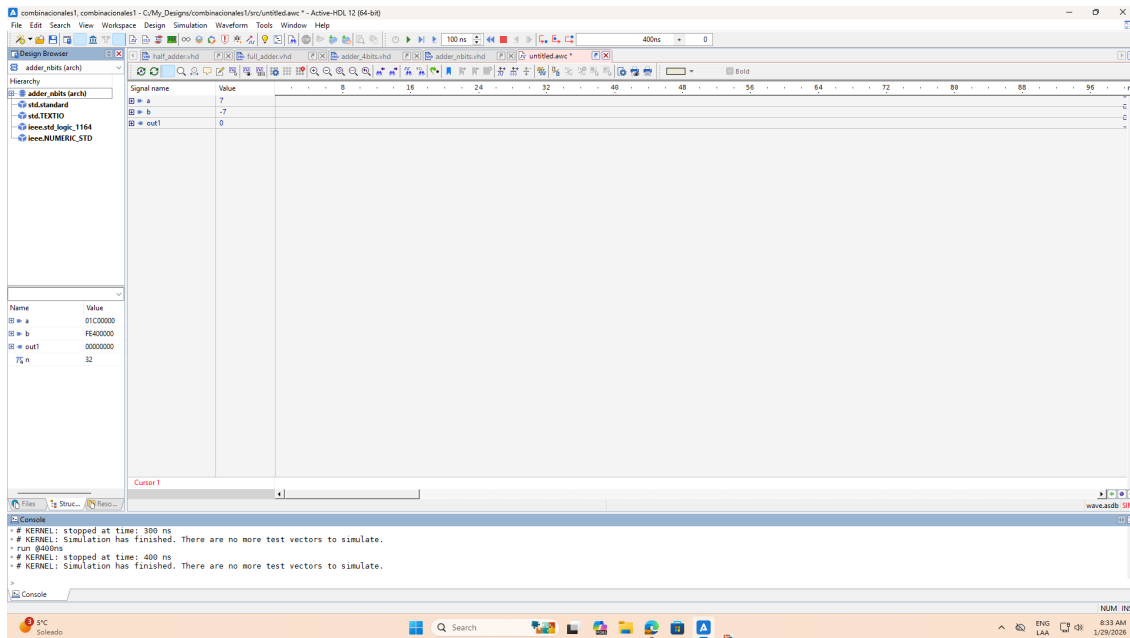
```

Listing 3: Código VHDL del sumador de 4 bits.

6.4. Módulo 4: Sumador de n bits con signo adder_nbits

Función. Suma dos valores con signo en complemento a dos.

Simulación.

Figura 5: Simulación del sumador de n bits con signo, caso 1.Figura 6: Simulación del sumador de n bits con signo, caso 2.

Código VHDL.

```
1  Library IEEE;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity adder_nbits is
6      generic(n: integer:=4);
7      port(
```

```

8   a, b: in std_logic_vector(n-1 downto 0);
9   out1: out std_logic_vector(n-1 downto 0)
10  );
11  end adder_nbits;
12
13  architecture arch of adder_nbits is
14  begin
15      out1 <= std_logic_vector(signed(a)+signed(b));
16  end arch;

```

Listing 4: Código VHDL del sumador de n bits con signo.

6.5. Módulo 5: Flip-Flop tipo D FFD

Función. Con $\text{rst}=1$ la salida se limpia a cero. Con $\text{rst}=0$ la salida copia a D en flanco de subida del reloj.

Simulación.

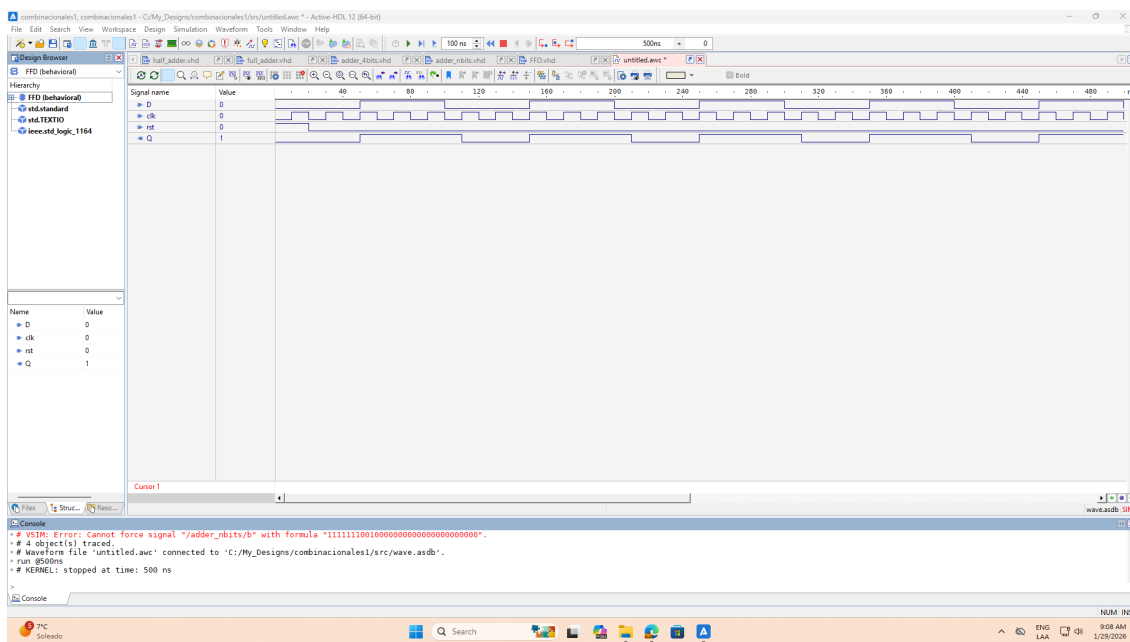


Figura 7: Simulación del flip-flop tipo D en Active-HDL.

Código VHDL.

```

1  Library IEEE;
2  use ieee.std_logic_1164.all;
3
4  entity FFD is
5      port(
6          D, clk, rst: in std_logic;
7          Q: out std_logic
8      );
9  end FFD;
10

```

```

11 architecture behavioral of FFD is
12 begin
13   process(clk,rst)
14   begin
15     if rst='1'then
16       Q<='0';
17     elsif rising_edge(clk) then
18       Q<=D;
19     end if;
20   end process;
21 end behavioral;

```

Listing 5: Código VHDL del flip-flop tipo D.

6.6. Módulo 6: Registro PIPO PIP0_register

Función. Con $en=1$ carga el vector D en flanco de subida. Con $en=0$ mantiene el valor. Con reset asíncrono se inicializa.

Casos medidos.

Cuadro 3: Casos de verificación del registro PIPO.

rst	en	D	Q esperado
1	0	xxxx	0000
0	1	1010	1010
0	0	0101	1010
0	1	0101	0101

Código VHDL.

```

1  Library IEEE;
2  use ieee.std_logic_1164.all;
3
4  entity PIP0_register is
5    generic(n: integer:=32;
6      initial: std_logic_vector:=x"00000001");
7    port(
8      clk, rst, en: in std_logic;
9      D: in std_logic_vector(n-1 downto 0);
10     Q: out std_logic_vector(n-1 downto 0)
11   );
12 end PIP0_register;
13
14 architecture behavioral of PIP0_register is
15   signal Qi:std_logic_vector(n-1 downto 0);
16   begin
17     Q<=Qi;
18     process(clk,rst,en)--"Lista de sensibilidad"
19     begin
20       if rst='1'then
21         Qi<=initial;

```

```
22     elsif rising_edge(clk) then
23         if en='1' then
24             Qi<=D;
25         else
26             Qi<=Qi;
27         end if;
28     end if;
29 end process;
30 end behavioral;
```

Listing 6: Código VHDL del registro PIPO.

7. Comparación teórico-práctica

En combinacionales, las salidas medidas en simulación coinciden con las tablas de verdad. En el medio sumador también se comparó simulación contra hardware, y las cuatro combinaciones dieron el mismo resultado en los LEDs. En secuenciales, el flip-flop y el registro siguen la idea de reset y captura con reloj, y eso se ve en el cambio de la salida en simulación.

8. Conclusiones

Se describieron módulos combinacionales y secuenciales en VHDL y se validaron con simulación en Active-HDL. En los combinacionales se revisaron tablas de verdad y casos con acarreo. En el sumador de 4 bits, la propagación del acarreo es lo que hace que funcione en todos los casos.

Por falta de tiempo se implementó en hardware únicamente el medio sumador, y las pruebas en la tarjeta coincidieron con la tabla de verdad. Como trabajo futuro, se puede implementar el resto de módulos en la Pynq Z1 y comparar hardware contra simulación.

9. Referencias

Referencias

- [1] O. Guillén Fernández. Práctica 0: Descripción de Circuitos Combinacionales y Secuenciales en VHDL. Universidad de las Américas Puebla, 2026.
- [2] IEEE. IEEE Standard VHDL Language Reference Manual.
- [3] Digilent. PYNQ-Z1 Reference Manual. Disponible en: <https://digilent.com/reference/programmable-logic/pynq-z1/start>

- [4] Aldec. Active-HDL Documentation. Disponible en: https://www.aldec.com/en/products/fpga_simulation/active_hdl
- [5] AMD Xilinx. Vivado Design Suite User Guide. Disponible en: <https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0002-vivado-design-hub.html>

